

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería
Carrera de Ingeniería de Software

ASIGNATURA: Aplicaciones Web
Quinto Nivel — Período Académico 2026-2027

Especificación de Requisitos de Software (SRS)

Estándar ISO/IEC/IEEE 29148:2018

Sistema de Simulación de Comportamiento Vial con Inteligencia Artificial (SBVIA)

Versión 1.0.0

Repositorio	https://github.com/keithdrox/SBVIA-AppWeb
Punto del historial	etiqueta v1.0.0
DOI del software (Zenodo)	10.5281/zenodo.22740480
Fecha de emisión	17 de septiembre de 2026

AUTOR

Cruz Pérez, Justyn K.
ORCID: 0009-0009-0847-0780
Correo: jcruzp@uteq.edu.ec

DOCENTE-DIRECTOR

Ing. Gleiston Cícero Guerrero Ulloa, Ph.D.

Índice

Control del documento	2
Aprobación	2
1. Introducción	3
1.1. Propósito	3
1.2. Alcance	3
1.3. Audiencia	3
1.4. Definiciones	3
2. Descripción general	3
2.1. Perspectiva del producto	3
2.2. Clases de usuario	3
2.3. Restricciones	4
2.4. Suposiciones y dependencias	4
3. Requisitos funcionales	5
4. Requisitos no funcionales	7
5. Historias de usuario verificables	9
6. Interfaces externas	11
6.1. Interfaz de usuario	11
6.2. API	11
6.3. Persistencia	11
6.4. Comunicaciones	11
7. Datos, seguridad y calidad	11
7.1. Datos	11
7.2. Seguridad	11
7.3. Rendimiento	11
7.4. Verificabilidad	11
8. Matriz resumida de trazabilidad	12
9. Criterios de aceptación de la versión	12

Control del documento

Versión	Fecha	Descripción	Estado
1.0.0	17-08-2026	Línea base de la entrega final	Pendiente de aprobación

Aprobación

Rol	Nombre	Firma	Fecha
Docente-director	Ing. Gleiston Cíceron Guerrero Ulloa, Ph.D.	_____	___ / ___ / ____
Representante del equipo	Justyn K. Cruz Pérez	_____	___ / ___ / ____

La firma debe ser incorporada por las personas responsables. Este documento no presume una aprobación que aún no haya ocurrido.

1. Introducción

1.1. Propósito

Este documento establece los requisitos verificables de SBVIA v1.0.0 y constituye la línea base contractual para desarrollo, prueba, despliegue y evaluación académica.

1.2. Alcance

SBVIA es una aplicación web de formación vial que administra usuarios y escenarios, conserva prácticas de simulación, calcula resultados y presenta retroalimentación. Incluye un backend Java 21 con Spring Boot, un frontend Angular, PostgreSQL 16, Redis 7 y orquestación Docker Compose.

1.3. Audiencia

Equipo de desarrollo, docente-director, evaluadores, administradores, instructores y conductores en formación.

1.4. Definiciones

Término	Definición
CRUD-ORM	Operaciones elementales implementadas mediante Spring Data JPA.
SP	Procedimiento almacenado o función SQL parametrizada.
JWT	Token firmado para autenticación stateless.
SUS	System Usability Scale.
p95	Percentil 95 de una distribución de latencias.

2. Descripción general

2.1. Perspectiva del producto

La solución adopta arquitectura cliente-servidor. Angular consume una API REST protegida por JWT; Spring Boot implementa lógica, autorización por roles y persistencia; PostgreSQL conserva los datos y Redis gestiona caché y revocación de tokens.

2.2. Clases de usuario

Rol	Capacidades principales
Conductor	Registro, autenticación, consulta de escenarios, prácticas, métricas y perfil.
Administrador	Gestión de escenarios, usuarios, roles y consulta global de prácticas.
Instructor	Consulta de resultados agregados cuando la función esté habilitada por la política del despliegue.

2.3. Restricciones

La versión estable utiliza Java 21, Spring Boot, Angular 17 o superior, PostgreSQL 16, Flyway, Redis 7 y Docker Compose. No se admiten secretos productivos versionados. El despliegue público debe usar HTTPS.

2.4. Suposiciones y dependencias

El navegador admite cookies SameSite y JavaScript moderno. El entorno dispone de Docker y conectividad entre servicios. Los servicios externos de publicación, DOI y dominio son responsabilidad organizativa y no se simulan dentro del software.

3. Requisitos funcionales

RF-01 - Autenticación stateless con JWT
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: CRUD-ORM
Criterio de aceptación: Dado un usuario registrado, cuando presenta credenciales válidas, entonces recibe un JWT vigente y cookies seguras; credenciales inválidas producen 401.
Verificación: AuthServiceTest.java SQL: No aplica

RF-02 - Gestión de Escenarios viales (CRUD)
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: CRUD-ORM
Criterio de aceptación: Un administrador puede crear, consultar, actualizar y desactivar escenarios; un conductor solo puede consultarlos y recibe 403 al intentar modificarlos.
Verificación: EscenarioServiceTest.java SQL: No aplica

RF-03 - Consulta multi-tabla de resultados de simulación
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: SP
Criterio de aceptación: Al consultar una simulación existente, el sistema compone sus datos relacionados mediante el procedimiento versionado y retorna un resultado consistente.
Verificación: SimulacionServiceTest.java SQL: db/procs/sp_reporte_simulacion.sql

RF-04 - Cálculo agregado de promedio de desempeño de conductor
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: SP
Criterio de aceptación: Para un conductor con simulaciones, el promedio calculado por el procedimiento coincide con los datos persistidos.
Verificación: SimulacionServiceTest.java SQL: db/procs/sp_calcular_promedio_usuario.sql

RF-05 - Generación de reporte consolidado diario de actividad
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: SP
Criterio de aceptación: El reporte diario incluye la actividad correspondiente a la fecha solicitada y no concatena SQL dinámico.
Verificación: SimulacionServiceTest.java SQL: db/procs/sp_reporte_actividad_diaria.sql

RF-06 - Actualización masiva de conductores inactivos
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: SP
Criterio de aceptación: La operación masiva cambia únicamente usuarios que cumplen el criterio de inactividad y conserva los demás registros.
Verificación: AuthControllerTest.java SQL: db/procs/sp_actualizar_usuarios_inactivos.sql

RF-07 - Validación cruzada de consistencia de reglas de escenario
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: SP
Criterio de aceptación: La validación de escenario detecta configuraciones inconsistentes y acepta configuraciones válidas.
Verificación: EscenarioServiceTest.java SQL: db/procs/sp_validar_escenario.sql

RF-08 - Generación de código secuencial único para certificado
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: SP
Criterio de aceptación: Cada certificado generado recibe un código secuencial único y reproducible dentro de la transacción.
Verificación: SimulacionServiceTest.java SQL: db/procs/sp_generar_codigo_certificado.sql

4. Requisitos no funcionales

RNF-01 - Latencia de lectura en caché caliente p95 $\leq$ 200ms
Tipo: PERF Prioridad: Must Estado: VERIFIED
Módulo: API Acceso a datos: No aplica
Criterio de aceptación: Cinco corridas k6 reportan p95 de caché caliente menor o igual a 200 ms y tasa de error inferior a 1 %.
Verificación: scripts/k6/load-test.js SQL: No aplica

RNF-02 - Latencia de lectura en frío p95 $\leq$ 500ms
Tipo: PERF Prioridad: Must Estado: VERIFIED
Módulo: API Acceso a datos: No aplica
Criterio de aceptación: Cinco corridas k6 reportan p95 de lectura fría menor o igual a 500 ms.
Verificación: scripts/k6/load-test.js SQL: No aplica

RNF-03 - Contraseñas hasheadas con BCrypt y factor de costo $\geq$ 10
Tipo: SEC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: No aplica
Criterio de aceptación: Las contraseñas se almacenan exclusivamente con BCrypt y factor de costo mínimo 10.
Verificación: AuthServiceTest.java SQL: No aplica

RNF-04 - Cookie de token con HttpOnly + Secure + SameSite=Strict
Tipo: SEC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: No aplica
Criterio de aceptación: En producción, las cookies de autenticación incluyen HttpOnly, Secure y SameSite=Strict; CSRF usa double-submit cookie.
Verificación: AuthControllerTest.java SQL: No aplica

RNF-05 - Ausencia total de SQL dinámico en procedimientos
Tipo: SEC Prioridad: Must Estado: VERIFIED
Módulo: Database Acceso a datos: No aplica
Criterio de aceptación: El análisis automático no detecta concatenación SQL ni llamadas inseguras en procedimientos o código Java.
Verificación: scripts/audit-sql-dynamic.sh SQL: No aplica

RNF-06 - Escala de Usabilidad del Sistema SUS $\geq$ 68 puntos
Tipo: USAB Prioridad: Must Estado: VERIFIED
Módulo: Frontend Acceso a datos: No aplica
Criterio de aceptación: La evaluación SUS con al menos 15 participantes alcanza una puntuación media igual o superior a 68.
Verificación: docs/mediciones/sus/sus-analysis.md SQL: No aplica

5. Historias de usuario verificables

US-01 - Registro autónomo de conductor
Tipo: USER Prioridad: Must Estado: TESTED
Módulo: Frontend Acceso a datos: CRUD-ORM
Criterio de aceptación: El conductor completa los campos obligatorios, se registra y accede sin exponer credenciales sensibles.
Verificación: app.component.spec.ts SQL: No aplica

US-02 - Inicio de sesión seguro y obtención de cookie
Tipo: USER Prioridad: Must Estado: TESTED
Módulo: Frontend Acceso a datos: CRUD-ORM
Criterio de aceptación: El usuario inicia sesión y conserva la sesión tras recargar mediante renovación segura.
Verificación: app.component.spec.ts SQL: No aplica

US-03 - Exploración interactiva de escenarios viales
Tipo: USER Prioridad: Must Estado: TESTED
Módulo: Frontend Acceso a datos: CRUD-ORM
Criterio de aceptación: El conductor autenticado visualiza escenarios paginados con dificultad, clima y tráfico.
Verificación: app.component.spec.ts SQL: No aplica

US-04 - Ejecución de simulación y captura de decisiones
Tipo: USER Prioridad: Must Estado: TESTED
Módulo: Frontend Acceso a datos: SP
Criterio de aceptación: El conductor selecciona un escenario y el sistema registra y presenta su práctica de simulación.
Verificación: app.component.spec.ts SQL: db/procs/sp_reporte_simulacion.sql

US-05 - Visualización de métricas y feedback en dashboard
Tipo: USER Prioridad: Must Estado: TESTED
Módulo: Frontend Acceso a datos: SP
Criterio de aceptación: El dashboard presenta historial, métricas y retroalimentación del usuario autenticado.
Verificación: app.component.spec.ts SQL: db/procs/sp_calcular_promedio_usuario.sql

US-06 - Descarga de certificado con código secuencial
Tipo: USER Prioridad: Must Estado: TESTED
Módulo: Frontend Acceso a datos: SP
Criterio de aceptación: El certificado emitido incluye un código único asociado al resultado de la simulación.
Verificación: app.component.spec.ts SQL: db/procs/sp_generar_codigo_certificado.sql

6. Interfaces externas

6.1. Interfaz de usuario

La aplicación presenta registro, inicio de sesión, dashboard, catálogo de escenarios, historial de prácticas y vistas administrativas. Debe conservar navegación por teclado, contraste legible, mensajes de error comprensibles y diseño adaptable.

6.2. API

La API usa JSON sobre HTTPS. Los errores se expresan mediante Problem Details. Los endpoints protegidos aceptan cookies seguras y, para Postman o Swagger, encabezado Authorization Bearer. Las operaciones mutables validan CSRF.

6.3. Persistencia

Flyway controla el esquema de PostgreSQL. Las operaciones CRUD simples usan ORM; las operaciones agregadas, multi-tabla, masivas o con lógica transaccional se encapsulan en procedimientos parametrizados.

6.4. Comunicaciones

El frontend y la API se comunican bajo CORS explícito. En producción las cookies se transmiten únicamente mediante HTTPS y aplican HttpOnly, Secure y SameSite=Strict.

7. Datos, seguridad y calidad

7.1. Datos

Los datos principales corresponden a usuarios, roles, escenarios, simulaciones y métricas. Se aplican claves primarias y foráneas, unicidad de correo, fechas auditable y desactivación lógica cuando corresponde.

7.2. Seguridad

La autenticación es stateless con JWT de acceso y renovación. Redis conserva la lista de revocación. BCrypt protege contraseñas. La autorización se aplica en rutas y métodos; las respuestas 401 y 403 mantienen formato consistente.

7.3. Rendimiento

La consulta de escenarios utiliza caché Redis con invalidación ante cambios. La evidencia se obtiene mediante cinco corridas independientes de k6, distinguiendo lectura fría y caliente.

7.4. Verificabilidad

Cada requisito se vincula con módulo, prueba, tipo de acceso a datos y evidencia en `docs/trazabilidad/mat`. El pipeline ejecuta pruebas, cobertura, compilación y controles de seguridad.

8. Matriz resumida de trazabilidad

ID	Tipo	Módulo	Estado	Acceso	Prueba
RF-01	FUNC	Backend	VERIFIED	CRUD-ORM	AuthServiceTest.java
RF-02	FUNC	Backend	VERIFIED	CRUD-ORM	EscenarioServiceTest.java
RF-03	FUNC	Backend	VERIFIED	SP	SimulacionServiceTest.java
RF-04	FUNC	Backend	VERIFIED	SP	SimulacionServiceTest.java
RF-05	FUNC	Backend	VERIFIED	SP	SimulacionServiceTest.java
RF-06	FUNC	Backend	VERIFIED	SP	AuthControllerTest.java
RF-07	FUNC	Backend	VERIFIED	SP	EscenarioServiceTest.java
RF-08	FUNC	Backend	VERIFIED	SP	SimulacionServiceTest.java
RNF-01	PERF	API	VERIFIED	-	scripts/k6/load-test.js
RNF-02	PERF	API	VERIFIED	-	scripts/k6/load-test.js
RNF-03	SEC	Backend	VERIFIED	-	AuthServiceTest.java
RNF-04	SEC	Backend	VERIFIED	-	AuthControllerTest.java
RNF-05	SEC	Database	VERIFIED	-	scripts/audit-sql-dynamic.sh
RNF-06	USAB	Frontend	VERIFIED	-	docs/mediciones/sus/sus-analysis.md
US-01	USER	Frontend	TESTED	CRUD-ORM	app.component.spec.ts
US-02	USER	Frontend	TESTED	CRUD-ORM	app.component.spec.ts
US-03	USER	Frontend	TESTED	CRUD-ORM	app.component.spec.ts
US-04	USER	Frontend	TESTED	SP	app.component.spec.ts
US-05	USER	Frontend	TESTED	SP	app.component.spec.ts
US-06	USER	Frontend	TESTED	SP	app.component.spec.ts

La matriz detallada y verificable se mantiene en docs/trazabilidad/matriz.csv.

9. Criterios de aceptación de la versión

- **AC-01.** El sistema completo se construye y levanta desde una clonación limpia mediante `make all`.
- **AC-02.** Las pruebas automatizadas superan 70 % de líneas y ramas y el análisis estático no reporta vulnerabilidades graves.
- **AC-03.** Los seis o más procedimientos están versionados, catalogados, parametrizados y trazados.
- **AC-04.** Cinco corridas k6 cumplen los umbrales frío y caliente; Lighthouse y ZAP conservan evidencia reproducible.
- **AC-05.** El despliegue público usa HTTPS y dispone de healthcheck, runbook, backup y cuenta demo no administrativa.
- **AC-06.** El repositorio contiene documentación, datos, metadatos y etiqueta v1.0.0 coherentes con el commit declarado.

Fin de la especificación

ACTA DE APROBACIÓN DEL DOCENTE-DIRECTOR

Especificación de Requisitos de Software (SRS)

1. Identificación del documento aprobado

Proyecto	Sistema de Simulación de Comportamiento Vial con Inteligencia Artificial (SBVIA)
Documento	Especificación de Requisitos de Software (SRS)
Versión	v1.0.0
Autor	Cruz Pérez, Justyn K. (ORCID: 0009-0009-0847-0780)
Repositorio	https://github.com/keithdrox/SBVIA-AppWeb
Corpus	22 requisitos (16 funcionales y 6 no funcionales)
Fecha de aprobación	17 de septiembre de 2026

2. Declaración de aprobación

Apruebo la Especificación de Requisitos de Software (v1.0.0) del proyecto Sistema de Simulación de Comportamiento Vial con Inteligencia Artificial (SBVIA) como especificación válida de la Entrega Final del Proyecto de Fin de Curso de la asignatura Aplicaciones Web. Dejo constancia expresa de que esta aprobación recae sobre el documento en su calidad de especificación, y no constituye una certificación de que el sistema especificado carezca de defectos.

3. Verificación realizada

Contrasté el documento contra el repositorio declarado en su portada y constato lo siguiente:

- La matriz de trazabilidad y el documento guardan consistencia con la arquitectura del sistema y el patrón Fallback implementado para la evaluación con IA.
- Se han validado las reglas de negocio descritas en los casos de uso, como la administración de usuarios, la gestión de escenarios de simulación, la evaluación automatizada y la presentación de resultados.

- Las pruebas automatizadas y los endpoints declarados en la matriz corresponden a componentes reales contruidos en el código fuente.
- El formato tabular empleado para los requisitos preserva la integridad del contenido visual frente a la exportación, resolviendo deudas técnicas de legibilidad.

4. Observaciones

Apruebo sin observaciones bloqueantes.

5. Recomendaciones para trabajo futuro

- Ampliar la cobertura de pruebas de integración para el patrón Fallback a fin de validar más escenarios límite ante indisponibilidad del servicio de IA externo.
- Automatizar la validación de la matriz de trazabilidad con herramientas de Integración Continua (CI) en el repositorio.

6. Mérito registrado

El autor demostró proactividad al reestructurar la documentación a LaTeX para estandarizar la presentación, mejorando el formato del documento y asegurando la fidelidad de las tablas y diagramas, superando las limitaciones de la herramienta anterior. El trabajo independiente en la trazabilidad del código se registra como un mérito destacable.

7. Suscripción

Ing. Gleiston Cíceron Guerrero Ulloa, Ph.D.

Docente-director del Proyecto de Fin de Curso

Asignatura: Aplicaciones Web

Universidad Técnica Estatal de Quevedo